

System Components & Security Architecture

Detailed breakdown of tier responsibilities, rendering engines, and security enforcement controls.

ARCHITECTURAL COMPONENTS

1. Presentation Tier

Frontend

- Framework:** Streamlit 1.65+
- Role:** Provides a clean, responsive single-page application for data entry, progress feedback, dark-mode document previews, inline editing, and binary file downloads.
- State Management:** Uses `st.session_state` to retain generated agreements, edited drafts, parsed terms, and uploaded branding across UI re-renders.

2. API Gateway Tier

Backend API

- Framework:** FastAPI with Starlette and Uvicorn
- Role:** Exposes stateless REST endpoints (`GET /`, `GET /health`, `POST /generate`).
- Validation:** Enforces strict typing, string length, and semantic date ordering using Pydantic V2 models.
- Security & Error Handling:** Intercepts unhandled exceptions with global handlers to prevent exposure of internal stack traces, system paths, or credential strings.

3. Artificial Intelligence Tier

AI Core

- Engine:** Google Gemini API via official SDK (`google-genai` and `google-generativeai`).
- Model:** Defaulting to `gemini-2.5-flash` with seamless fallback to `gemini-1.5-pro` or `gemini-1.5-flash` .
- Prompt Engineering:** Enforces legal drafting principles, avoids hallucinated statutory citations, generates structured contractual provisions, and places bracketed placeholders for omitted jurisdiction-specific items.

4. Text Processing & Sanitization Tier

Data Processing

- Module:** `backend/utils/sanitization.py`
- Role:** Normalizes typographic quotes, eliminates backtick markdown fencing, cleans irregular unicode characters, and converts semicolon-separated terms into structured key-value pairs for the Terms Table.

DOCUMENT RENDERING TIER

The system implements multi-format export capabilities using specialized rendering engines optimized for distinct file outputs:

Format Generator	Engine / Library	Key Responsibilities & Capabilities
TXT Generator	Standard I/O	Generates clean, UTF-8 encoded plain text agreements for lightweight storage and basic editing.
DOCX Generator	<code>python-docx</code>	Constructs Microsoft Word files with Times New Roman typography, embedded company logos, shaded Schedule A tables, and formal signature lines.
PDF Generator	<code>reportlab</code>	Utilizes a custom <code>NumberedCanvas</code> to draw multi-page headers, footers ("Page X of Y"), legal disclaimers, table borders, and signature boxes.

SECURITY ARCHITECTURE

1. Environment Isolation API credentials are loaded strictly from <code>.env</code> or system environment variables; no secrets are ever committed or hard-coded into repository source files.	2. Input Sanitization All user inputs undergo strict length constraints, structural type checks, and regex sanitization to prevent injection vulnerabilities across backend handlers.
3. Logo Image Verification Uploaded branding files are validated via Pillow's <code>verify()</code> against MIME types, maximum file size limits (5MB), and structural corruption.	4. Data Privacy No user contract data is logged to disk or exposed in server execution logs, maintaining complete operational confidentiality for generated agreements.